



FINAL%

Faculty of Science, Agriculture and Engineering

ASSIGNMENT COVER SHEET

MODULE TITLE	SOFTWARE ENGINEERING	
MODULE CODE	5EEE 491	
LECTURER NAME	Dr.Oyebode	
Topic	Recipe Management System	
DUE DATE	10 May 2026	
NON - PLAGIARISM DECLARATION I know that plagiarism means taking and using the ideas, writings, works or inventions of another as if they were one's own. I know that plagiarism not only includes verbatim copying, but also the extensive use of another person's ideas without proper acknowledgement (which includes the proper use of quotation marks). I know that plagiarism covers this sort of use of material found in textual sources and from the Internet. I acknowledge and understand that plagiarism is wrong. I understand that my research must be accurately referenced. I have followed the rules and conventions concerning referencing, citation and the use of quotations as set out in the Departmental Guide. This assignment is my own work, or my group's own unique group assignment. I acknowledge that copying someone else's assignment, or part of it, is wrong, and that submitting identical work to others constitutes a form of plagiarism. I have not allowed, nor will I in the future allow, anyone to copy my work with the intention of passing it off as their own work. By signing this cover sheet, I agree that I have read and understood the above. I acknowledge that should it be found to be higher than the acceptable similarity percentage, I may receive 0 (ZERO) for my assignment.		
STUDENT NAME	STUDENT NO	
N Zungu	202375568	
LECTURER M REMARKS		

1 AI Tool Use Disclosure

For each AI tool that you have used, provide a brief, high-level summary:

1.	Tool Name and Version:	DeepSeek
	Purpose:	finding solution for errors
	Scope of Use	

2.	Tool Name and Version:	ChatGBT
	Purpose:	For finding the correct structure and code for report
	Scope of Use	

**Add as many tables as you require or delete tables as appropriate.*

1. AI Tool Use Detailed Disclosure by Category

For every instance where you used an AI tool, you must provide a detailed explanation.

A. AI as a Writing Tool (e.g., drafting, summarising, paraphrasing, editing, proofreading, image generation)

<i>Requirement</i>	<i>Explanation</i>
<i>Specific use</i>	C# codes
<i>Input/Prompt</i>	<i>prompt</i>
<i>Validation process</i>	
<i>Required evidence</i>	

Contents

List of Figures.....	5
----------------------	---

1. Introduction	6
2. Problem Statement	6
3. Project Objectives	7
4. Requirements Analysis.....	7
4.1. Functional Requirements.....	7
4.2 Non-Functional Requirements	8
4.3 Stakeholder Analysis	8
5. GA2 Alignment	9
5.1 Application of Natural Sciences Knowledge.....	9
5.2 Use of Mathematics and Analytical Techniques	9
5.3 Application of Engineering Fundamentals.....	9
5.4 Engineering Specialisation Knowledge.....	9
5.5 Modelling, Analysis and Problem Conceptualisation	9
6. System Design	10
6.1 System Architecture.....	10
6.2 Database Design	10
6.3 Entity Relationship Explanation	11
6.4 User Interface Design	11
7. Technology Justification	13
7.1 ASP.NET Core MVC	13
7.2 C#.....	13
7.3 Entity Framework Core	13
7.4 Code-First Approach.....	13
7.5 SQL Server LocalDB	13
7.6 Bootstrap	13
8. Implementation.....	14
8.1 Development Methodology	14
9. Code Snippets and Explanation	14
9.1 Recipe Model.....	14
9.2 Ingredient Model	15
9.3 RecipeIngredient Model.....	16

9.4 Instruction Model	16
9.5 MealPlan Model	17
9.6 Review Model	17
9.7 AppDbContext	18
9.8 Program.cs Database Registration	19
9.9 Connection String	19
9.10 Recipe Search Code.....	19
9.11 Image Upload Code	20
9.12 Create Recipe Save Code	20
9.13 Shopping List Aggregation Code	21
9.14 Rating Average Code.....	21
9.15 Navigation Code	21
9.16 Back Button Code.....	22
10. Testing and Validation	22
10.2 Recipes Used for Testing	23
10.3 Bug Tracking and Fixes	23
11.Validation Against Project Requirements	23
12. Professional Considerations.....	24
12.1 Ethical Considerations	24
12.2 Security Measures	24
12.3 POPIA Compliance	24
12.4 Scalability	24
12.5 Maintainability	25
13. Natural Science, Data Flow and System Behaviour	25
14. Mathematical and Analytical Techniques.....	25
14.1 Rating Average	25
14.2 Shopping List Grouping	25
14.3 Algorithm Complexity	25
15. Limitations	26
16. Future Improvements	26
17. Conclusion	26

References.....	27
-----------------	----

List of Figuers

Figure 1 The system uses the MVC architecture.	10
Figure 2 Recipe Home Page	11
Figure 3 Add Recipe Page	12
Figure 4 Recipe Details Page	12
Figure 5 Meal Planner Page	13
Figure 6 Shopping List Page	13
Figure 7 This code uses data annotations such as [Required] and [MaxLength]. These help validate user input before saving it.	15
Figure 8 This model allows one ingredient to be reused in many recipes.	15
Figure 9 This bridge table is important because a recipe can have many ingredients, and an ingredient can appear in many recipes.	16
Figure 10 The StepNumber field helps display cooking instructions in the correct order.	16
Figure 11 This model links a recipe to a date and meal type such as Breakfast, Lunch, Dinner, or Snack.	17
Figure 12 Each DbSet becomes a database table. This follows the Code-First method where model classes are converted into database tables using migrations.	18
Figure 13 This allows ASP.NET Core to provide ApplicationDbContext automatically to controllers.	18
Figure 14 During testing, a SQL Server timeout error occurred. The problem was fixed by changing from localhost\SQLEXPRESS to LocalDB.	19
Figure 15 This improves usability because users can quickly find recipes	19
Figure 16 A unique file name is used to avoid overwriting images with the same name.	20
Figure 17 The recipe must be saved first so that it gets a Recipeld.	20
Figure 18 This is one of the main algorithms in the project. It collects ingredients from all planned recipes and groups them by ingredient name.	21
Figure 19 This prevents an error when a recipe has no reviews.	21
Figure 20 A small mistake in this file caused the Recipes link to display incorrectly. The issue was fixed by adding the missing <a> tag correctly.	21
Figure 21 This was added to the Meal Planner and Shopping List pages.	21

1. Introduction

A lot of people just keep their recipes in notebooks or their phone notes or maybe screenshots from social media. But when you got a lot of them it gets real messy and it's hard to find stuff. Like if you want to find a recipe that uses chicken or something, good luck with that. And also people make meal plans for the week but then they forget what stuff they need to buy at the store. So that's why we made this Recipe Management System to fix all that.

With this system you can add your recipes, put up photos of the food, write down the ingredients, write the cooking steps, search for recipes, make a meal plan, and it makes the shopping list for you automatically. So it makes cooking and planning meals way more easier and organized.

We built this project using ASP.NET Core MVC. MVC basically splits everything into three parts - the Model, the View, and the Controller. The Model does the data stuff and checks if everything's correct. The View is what the user sees on the screen. And the Controller handles what happens when the user clicks stuff. This way it's easier to fix things later and add new stuff. Our lecture notes said MVC helps keep the code organized and makes it easier to maintain and scale up because every part has its own job to do [1],[2].

We also used Entity Framework Core with the Code-First way. In this way you first write the C# model classes and then Entity Framework Core makes the database tables for you using migrations. We used migrations to create the recipe database and all the tables we needed [1],[2].

2. Problem Statement

The problem is recipes, meal plans and shopping lists are usually kept separate. A user might have recipes in one place, meal plans written somewhere else, and then they gotta manually write a shopping list. This wastes a lot of time and sometimes you forget ingredients when you're cooking. It's annoying.

This project fixes that by making one web system where you can organize recipes, plan meals, generate shopping lists, and review recipes all together. The official Project 4 description says the system needs to organize recipes, plan meals, generate shopping lists, and share recipes with others. It also needs stuff like adding recipes with ingredients, step by step instructions, uploading images, search and filtering, meal planning, shopping list generation, and rating and review functions [1],[2].

3. Project Objectives

the main goal of this project is to design and build a working Recipe Management System using ASP.NET Core MVC.

the specific objectives are:

1. To create a web application that stores recipes.
2. let users add ingredients and how much of each thing they need.
3. let users put in step-by-step cooking instructions.
4. Let user upload recipe photos and show them on the page.
5. let users search recipes by typing the name, picking a category, or looking for a specific ingredient.
6. To create a meal planning calendar.
7. To automatically generate a shopping list from planned meals.
8. To allow users to rate and review recipes.
9. To apply MVC architecture and Entity Framework Core Code-First development.
10. To test the system and fix errors found during development.

4. Requirements Analysis

4.1. Functional Requirements

functional requirements is basically what the system gotta do. for this project, the user must be able to do these things:

FR1: Add a recipe

user gotta be able to throw in a new recipe. they need to fill in the name, what kinda food it is, a quick description, a photo, the stuff you need, how much of each thing, and the steps to make it.

FR2: View recipes

on the home page they should see all there saved recipes.

FR3: View recipe details

when you click on one recipe you should see all the stuff in it - ingredients, steps, the photo, and reviews from other people.

FR4: Edit a recipe

user must be able to change anything in a recipe even after saving it.

FR5: Delete a recipe

user gotta be able to get rid of a recipe from the system if they dont want it no more.

FR6: Search and filter recipes

people should be able to search by typing the recipe name, or pick a category, or look up a specific ingredient they got in mind.

FR7: Upload recipe image

the system needs to let users put up a photo for there recipe.

FR8: Create meal plan

user should pick a recipe, choose a day, and say if its breakfast lunch or dinner.

FR9: Generate shopping list

the system should automaticly pull all the ingredients from the meal plan and spit out a shopping list.

FR10: Add reviews and ratings

users need to be able to put in their name, give a star rating, and drop a comment about the recipe.

4.2 Non-Functional Requirements

Non-functional requirements describe how well the system should work.

NFR1: Usability

app should be dead simple to use. buttons like Add Recipe, Edit, Delete, Meal Planner and Shopping List need to be right there so folks know what each one does straight away without having to figure it out.

NFR2: Reliability

system needs to save recipes in the database right and not lose nothing. if someone puts in a recipe it better still be there when they come back later.

NFR3: Maintainability

code gotta be kept in models, views and controllers all separate. that way its easier to fix things or add new stuff later without messing up what already works.

NFR4: Performance

recipe lists and search gotta load quick. nobody wants to sit around waiting for the page to show up.

NFR5: Security

need validation and anti-forgery on forms so people cant send fake stuff or try hack the system.

NFR6: Scalability

should build it so you can add more stuff later like login, sharing recipes, making PDFs, admin controls without having to start over from scratch.

NFR7: Data Integrity

fields like recipe title, category, ingredient name, instructions, reviewer name and rating gotta be checked. cant let empty or bad data get through.

4.3 Stakeholder Analysis

The main stakeholders:

User / Home Cook:

This is the person who adds recipes, plans meals, and checks the shopping list.

Student Developer:

The student designs, builds, tests, and explains the system.

Lecturer / Assessor:

The lecturer checks if the system meets the project requirements and GA2 outcomes.

Future Users:

These may be family members, students, or small food businesses that need to organize recipes.

5. GA2 Alignment

This project supports Graduate Attribute 2 because it uses software engineering knowledge to design and build a working system. GA2 is about coming up with creative solutions for complex engineering problems and making systems that meet what people actually need [1].

5.1 Application of Natural Sciences Knowledge

The system shows how software reacts when users do stuff, like sending HTTP requests, doing database queries, and saving data. for example, when someone fills out a recipe form, the data goes from the Razor View to the Controller and then into the database through Entity Framework Core [1]. boolean checks like `ModelState.IsValid`, `id == null`, and `imageFile != null` are used to control what the system does.

5.2 Use of Mathematics and Analytical Techniques

The system uses analytical thinking for search, filtering, calculating rating averages, and putting together shopping lists. for example, the recipe rating is worked out by taking the average of all ratings [3]. the shopping list uses grouping to collect ingredients from planned meals. the system also uses basic reliability reasoning during testing.

simple rating formula: $\text{Average Rating} = \text{Sum of all ratings} / \text{Number of ratings}$

5.3 Application of Engineering Fundamentals

The project follows software engineering steps like requirements analysis, design, implementation, testing, and improvement. it also uses the MVC design pattern, separation of concerns, model validation, database relationships, and modular code organization.

5.4 Engineering Specialisation Knowledge

the project uses ASP.NET Core MVC, C#, Razor Views, Entity Framework Core, SQL Server LocalDB, migrations, controllers, and data annotations [1]. the guide says you need implementation knowledge like model design, DbContext configuration, database migrations, CRUD controller actions, Razor views, and security stuff like `[ValidateAntiForgeryToken]` and `ModelState` validation [1],[2].

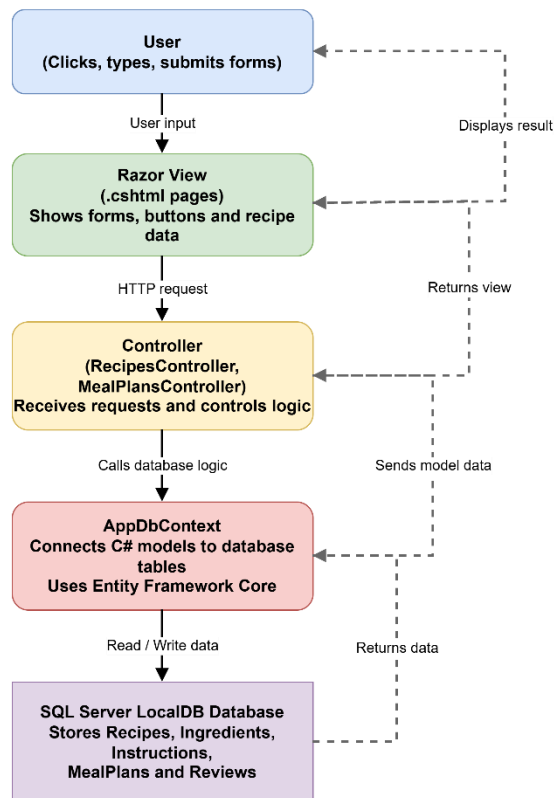
5.5 Modelling, Analysis and Problem Conceptualisation

The system got modelled using entities like Recipe, Ingredient, RecipeIngredient, Instruction, MealPlan, and Review. these entities form the database schema. The system also uses relationships like one-to-many and many-to-many.

6. System Design

6.1 System Architecture

Recipe Management System - MVC Data Flow Diagram



Example: When the user adds a recipe, the form data moves from the Razor View to the Controller. The Controller validates the data, then AppDbContext saves it into SQL Server LocalDB.

Figure

1 The system uses the MVC architecture.

The user messes with the view. the controller gets the request, handles the data, and uses AppDbContext to talk to the database. Then it sends the result back to the view.

6.2 Database Design

The main database tables are:

1. Recipes
2. Ingredients
3. RecipeIngredients
4. Instructions
5. MealPlans
6. Reviews

6.3 Entity Relationship Explanation

A recipe can have many instructions.

A recipe can have many reviews.

A recipe can appear in many meal plans.

A recipe can have a lot of ingredients in it.

One ingredient can be used in a lot of different recipes.

Because of that, the system uses a bridge table called RecipeIngredient to link them together.

6.4 User Interface Design

The system has the following main pages:

Recipes Page:

Shows all recipes as cards. It also has a search box and category filter.

Create Recipe Page:

Lets the user put in the recipe name, what type it is, a description, a photo, the stuff you need, how much of each thing, and the steps to cook it.

Recipe Details Page:

Shows full recipe information and reviews.

Meal Planner Page:

Shows planned meals with date, meal type, and recipe name.

Shopping List Page:

Shows ingredients automatically collected from planned meals.

Review Page:

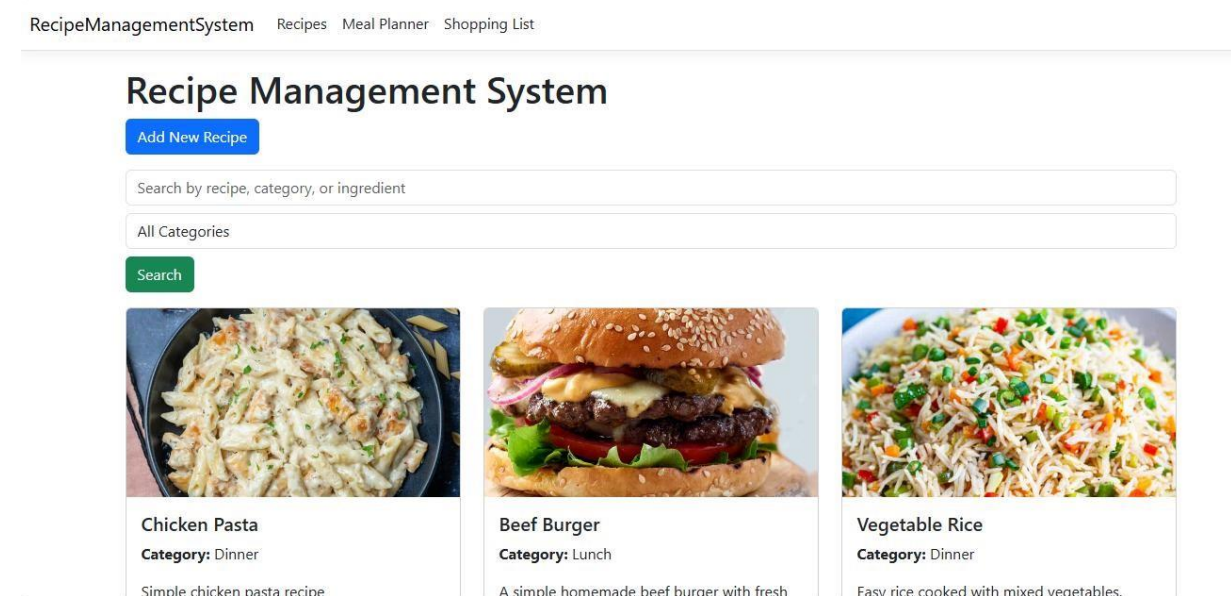


Figure 2 Recipe Home Page

Add Recipe

Recipe Title

Category

Description

Recipe Image

Choose File No file chosen

Ingredients

Ingredient name	Quantity e.g. 2 cups
Ingredient name	Quantity e.g. 2 cups
Ingredient name	Quantity e.g. 2 cups
Ingredient name	Quantity e.g. 2 cups
Ingredient name	Quantity e.g. 2 cups

Instructions

Step 1

Step 2

Step 3

Step 4

Step 5

Save Recipe Back

Figure 3 Add Recipe Page

Category: Dinner

Description: Simple chicken pasta recipe

Ingredients

- 2 cups - Pasta
- 500g - Chicken
- 1 teaspoon - Salt

Instructions

- Boil the pasta
- Cook the chicken
- Mix everything together

Reviews

Add Review

No reviews yet.

Edit Back

Figure 4 Recipe Details Page

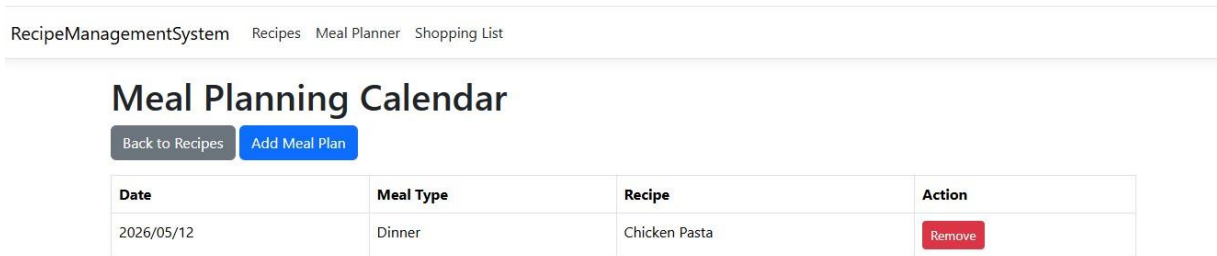


Figure 5 Meal Planner Page

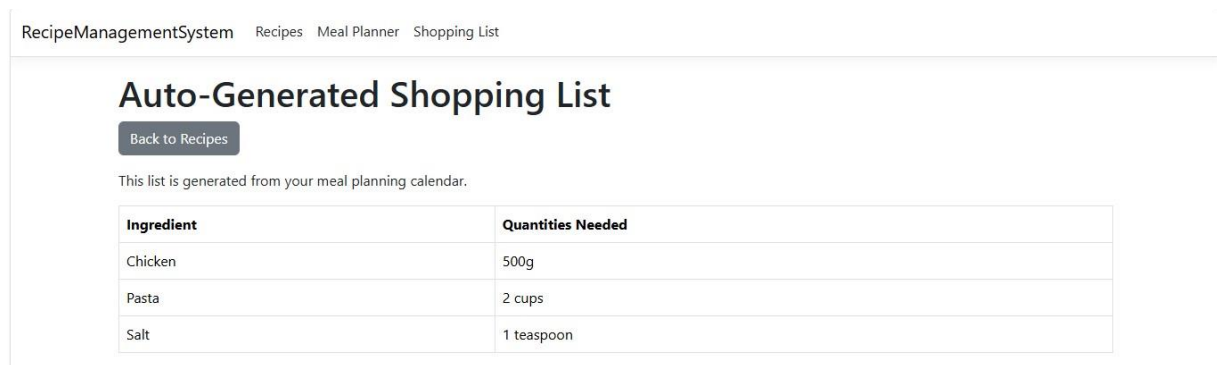


Figure 6 Shopping List Page

7. Technology Justification

7.1 ASP.NET Core MVC

I went with ASP.NET Core MVC cause it's solid for web apps and keeps things tidy. it breaks the app into Models, Views and Controllers which makes the code easier to figure out and change when needed.

7.2 C#

Picked C# cause it fits ASP.NET Core real well and it's a proper object-oriented language so the code stays neat and organised.

7.3 Entity Framework Core

used Entity Framework Core so we could work with the database through C# classes instead of messing with raw SQL. EF Core's an ORM so database stuff feels like normal coding with objects.

7.4 Code-First Approach

went Code-First cause we wanted the database tables built from our model classes. once the models were sorted we ran migrations to generate the schema.

7.5 SQL Server LocalDB

chose SQL Server LocalDB cause it works smooth with Visual Studio and is dead easy to set up. perfect for student work and local development.

7.6 Bootstrap

threw in Bootstrap for styling cause it gets buttons, forms, tables, cards and navigation looking alright quick without wasting time on custom CSS.

8. Implementation

8.1 Development Methodology

The project used a simple iterative development method. The system was built step by step. First, the models were created. Then the database context was created. After that, migrations were run. Controllers and views were then added. Finally, testing was done by adding recipes and checking the meal planner and shopping list[4].

The development steps were:

1. Create ASP.NET Core MVC project.
2. Install Entity Framework Core packages.
3. Create model classes.
4. Create ApplicationDbContext.
5. Add database connection string.
6. Register database service in Program.cs.
7. Run migrations.
8. Create controllers.
9. Create Razor views.
10. Test the system.
11. Fix navigation and database connection errors.

9. Code Snippets and Explanation

9.1 Recipe Model

The Recipe model stores the main recipe information. It also connects to ingredients, instructions, meal plans, and reviews.

```

RecipeManagementSystem
RecipeManagementSystem.Models.Recipe
1 using System.ComponentModel.DataAnnotations;
2
3 namespace RecipeManagementSystem.Models
4 {
5     17 references
6     public class Recipe
7     {
8         17 references
9         public int RecipeId { get; set; }
10
11         [Required]
12         [MaxLength(100)]
13         [DisplayName = "Recipe Title"]
14         14 references
15         public string Title { get; set; } = "";
16
17         [Required]
18         [MaxLength(50)]
19         13 references
20         public string Category { get; set; } = "";
21
22         [MaxLength(500)]
23         9 references
24         public string Description { get; set; } = "";
25
26         [DisplayName = "Image"]
27         8 references
28         public string? ImagePath { get; set; }
29
30         [DisplayName = "Created Date"]
31         0 references
32         public DateTime CreatedDate { get; set; } = DateTime.Now;
33
34         12 references
35         public List<RecipeIngredient> RecipeIngredients { get; set; } = new();
36         8 references
37         public List<Instruction> Instructions { get; set; } = new();
38         2 references
39         public List<MealPlan> MealPlans { get; set; } = new();
40         8 references
41         public List<Review> Reviews { get; set; } = new();
42     }
43 }

```

Figure 7 This code uses data annotations such as `[Required]` and `[MaxLength]`. These help validate user input before saving it.

9.2 Ingredient Model

The ingredient model stores ingredient names.

```

1  using System.ComponentModel.DataAnnotations;
2
3  namespace RecipeManagementSystem.Models
4  {
5      1 reference
      public class Ingredient
6      {
7          0 references
          public int IngredientId { get; set; }
8
9          [Required]
          [MaxLength(100)]
          0 references
          public string Name { get; set; } = "";
10
          0 references
          public List<RecipeIngredient> RecipeIngredients { get; set; } = new();
11
12      }
13
14  }
15

```

Figure 8 This model allows one ingredient to be reused in many recipes.

9.3 RecipeIngredient Model

This model creates the many-to-many relationship between recipes and ingredients.

```

1  using System.ComponentModel.DataAnnotations;
2
3  namespace RecipeManagementSystem.Models
4  {
5      2 references
      public class RecipeIngredient
6      {
7          0 references
          public int RecipeIngredientId { get; set; }
8
9          0 references
          public int RecipeId { get; set; }
          0 references
          public Recipe? Recipe { get; set; }
10
          0 references
          public int IngredientId { get; set; }
          0 references
          public Ingredient? Ingredient { get; set; }
11
          [Required]
          [MaxLength(50)]
          public string Quantity { get; set; } = "";
12
13      }
14
15  }
16

```

Figure 9 This bridge table is important because a recipe can have many ingredients, and an ingredient can appear in many recipes.

9.4 Instruction Model

The Instruction model stores cooking steps.

```

RecipeManagementSystem
RecipeManagementSystem.Models.Instruction
1 using System.ComponentModel.DataAnnotations;
2
3 namespace RecipeManagementSystem.Models
4 {
5     public class Instruction
6     {
7         public int InstructionId { get; set; }
8
9         public int RecipeId { get; set; }
10        ~ references
11        public Recipe? Recipe { get; set; }
12
13        [Required]
14        [Display(Name = "Step Number")]
15        0 references
16        public int StepNumber { get; set; }
17
18        [Required]
19        [MaxLength(1000)]
20        [Display(Name = "Instruction")]
21        ~ references
22        public string Description { get; set; } = "";
23    }
24 }

```

Figure 10 The StepNumber field helps display cooking instructions in the correct order.

9.5 MealPlan Model

The MealPlan model stores meal planning data.

```

RecipeManagementSystem
RecipeManagementSystem.Models.MealPlan
4 {
5     2 references
6     public class MealPlan
7     {
8         0 references
9         public int MealPlanId { get; set; }
10
11         [Required]
12         [Display(Name = "Meal Date")]
13         6 references
14         public DateTime MealDate { get; set; }
15
16         [Required]
17         [MaxLength(30)]
18         [Display(Name = "Meal Type")]
19         5 references
20         public string MealType { get; set; } = "";
21
22         3 references
23         public int RecipeId { get; set; }
24         6 references
25         public Recipe? Recipe { get; set; }
26     }
27 }

```

Figure 11 This model links a recipe to a date and meal type such as Breakfast, Lunch, Dinner, or Snack.

9.6 Review Model

The Review model stores recipe ratings and comments.

```

RecipeManagementSystem
RecipeManagementSystem.Models.Review
1 using System.ComponentModel.DataAnnotations;
2
3 namespace RecipeManagementSystem.Models
4 {
5     8 references
6     public class Review
7     {
8         2 references
9         public int ReviewId { get; set; }
10
11         5 references
12         public int RecipeId { get; set; }
13         1 reference
14         public Recipe? Recipe { get; set; }
15
16         [Required]
17         [MaxLength(100)]
18         [Display(Name = "Reviewer Name")]
19         5 references
20         public string ReviewerName { get; set; } = "";
21
22         [Required]
23         [Range(1, 5)]
24         6 references
25         public int Rating { get; set; }
26
27         [MaxLength(500)]
28         4 references
29         public string Comment { get; set; } = "";
30
31         2 references
32         public DateTime ReviewDate { get; set; } = DateTime.Now;
33     }
34 }
% No issues found

```

9.7 AppDbContext

AppDbContext connects the application to the database.

```

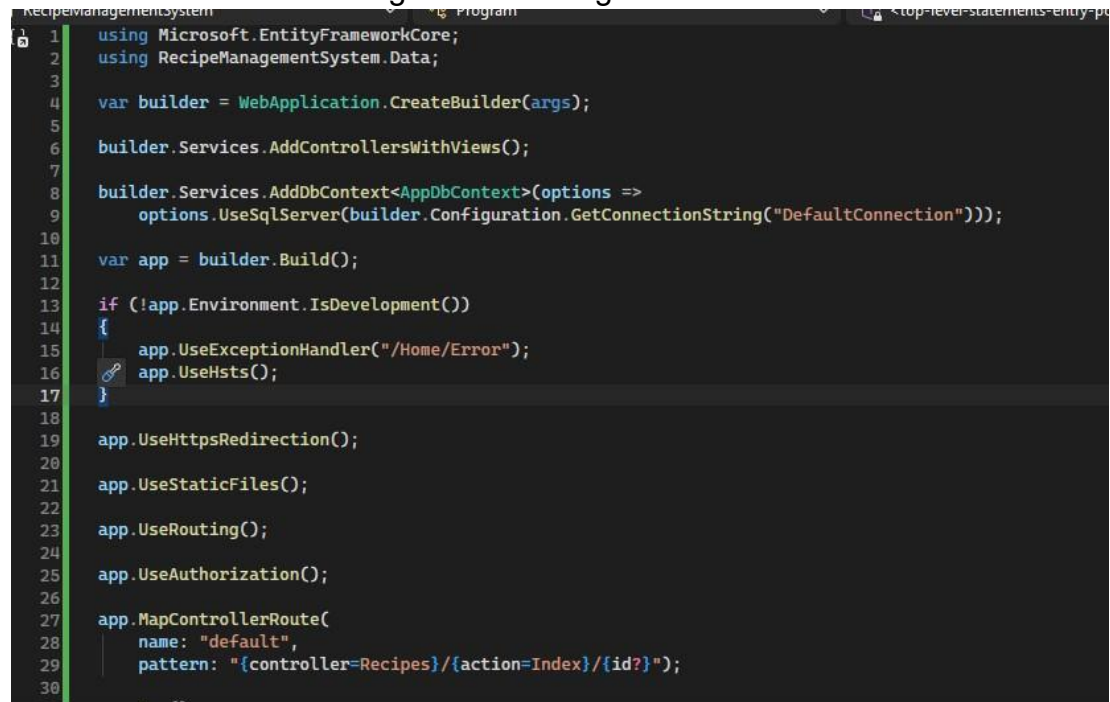
RecipeManagementSystem
RecipeManagementSystem.Data.AppDbContext
1 using Microsoft.EntityFrameworkCore;
2 using RecipeManagementSystem.Models;
3
4 namespace RecipeManagementSystem.Data
5 {
6     13 references
7     public class AppDbContext : DbContext
8     {
9         0 references
10         public AppDbContext(DbContextOptions<AppDbContext> options)
11             : base(options)
12         {
13         }
14
15         11 references
16         public DbSet<Recipe> Recipes { get; set; }
17         4 references
18         public DbSet<Ingredient> Ingredients { get; set; }
19         4 references
20         public DbSet<RecipeIngredient> RecipeIngredients { get; set; }
21         4 references
22         public DbSet<Instruction> Instructions { get; set; }
23         6 references
24         public DbSet<MealPlan> MealPlans { get; set; }
25         5 references
26         public DbSet<Review> Reviews { get; set; }
27     }
28 }

```

Figure 12 Each DbSet becomes a database table. This follows the Code-First method where model classes are converted into database tables using migrations.

9.8 Program.cs Database Registration

The database context is registered in Program.cs.



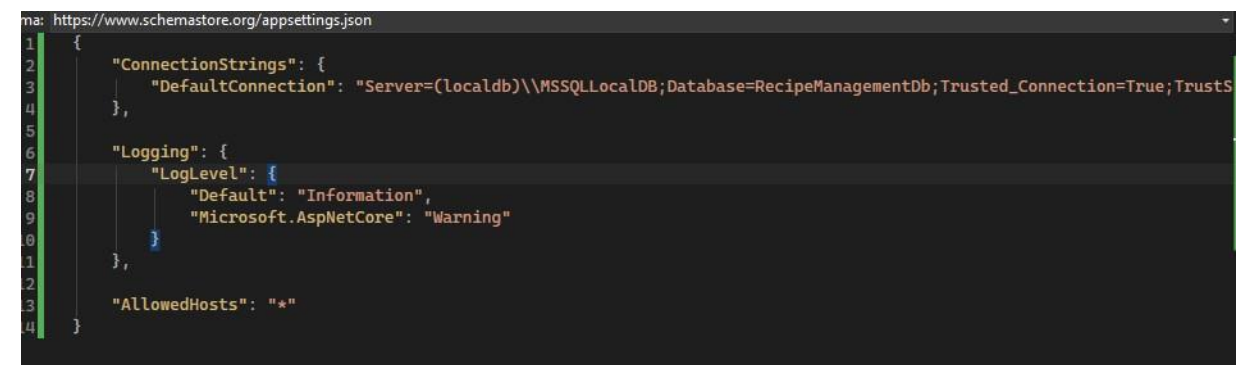
```
1 using Microsoft.EntityFrameworkCore;
2 using RecipeManagementSystem.Data;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddControllersWithViews();
7
8 builder.Services.AddDbContext<AppDbContext>(options =>
9     options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
10
11 var app = builder.Build();
12
13 if (!app.Environment.IsDevelopment())
14 {
15     app.UseExceptionHandler("/Home/Error");
16     app.UseHsts();
17 }
18
19 app.UseHttpsRedirection();
20
21 app.UseStaticFiles();
22
23 app.UseRouting();
24
25 app.UseAuthorization();
26
27 app.MapControllerRoute(
28     name: "default",
29     pattern: "{controller=Recipes}/{action=Index}/{id?}");
30
```

Figure

13 This allows ASP.NET Core to provide AppDbContext automatically to controllers.

9.9 Connection String

The final connection string used LocalDB to avoid SQL Server timeout problems.

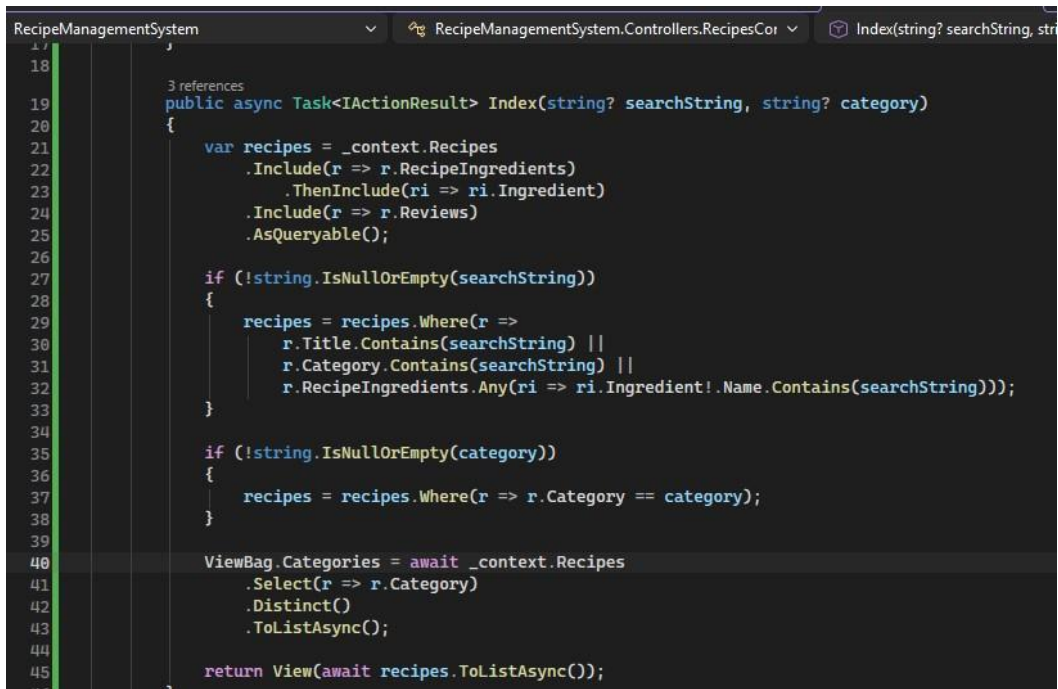


```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=RecipeManagementDb;Trusted_Connection=True;TrustS
4   },
5
6   "Logging": {
7     "LogLevel": {
8       "Default": "Information",
9       "Microsoft.AspNetCore": "Warning"
10    },
11  },
12
13  "AllowedHosts": "*"
14 }
```

Figure 14 During testing, a SQL Server timeout error occurred. The problem was fixed by changing from localhost\\SQLEXPRESS to LocalDB.

9.10 Recipe Search Code

The search function allows users to search by recipe title, category, or ingredient.



```

18
19 3 references
20 public async Task<IActionResult> Index(string? searchString, string? category)
21 {
22     var recipes = _context.Recipes
23         .Include(r => r.RecipeIngredients)
24         .ThenInclude(ri => ri.Ingredient)
25         .Include(r => r.Reviews)
26         .AsQueryable();
27
28     if (!string.IsNullOrEmpty(searchString))
29     {
30         recipes = recipes.Where(r =>
31             r.Title.Contains(searchString) ||
32             r.Category.Contains(searchString) ||
33             r.RecipeIngredients.Any(ri => ri.Ingredient!.Name.Contains(searchString)));
34     }
35
36     if (!string.IsNullOrEmpty(category))
37     {
38         recipes = recipes.Where(r => r.Category == category);
39     }
40
41     ViewBag.Categories = await _context.Recipes
42         .Select(r => r.Category)
43         .Distinct()
44         .ToListAsync();
45
46     return View(await recipes.ToListAsync());

```

Figure 15 This improves usability because users can quickly find recipes

9.11 Image Upload Code

The system saves uploaded recipe images inside wwwroot/images/recipe

```

if (imageFile != null && imageFile.Length > 0)
{
    string uploadsFolder = Path.Combine(
        _environment.WebRootPath, "images", "recipes");

    string uniqueFileName = Guid.NewGuid().ToString() + "_" + imageFile.FileName;
    string filePath = Path.Combine(uploadsFolder, uniqueFileName);

    using (var fileStream = new FileStream(filePath, FileMode.Create))
    {
        await imageFile.CopyToAsync(fileStream);
    }

    recipe.ImagePath = "/images/recipes/" + uniqueFileName;
}

```

Figure 16 A unique file name is used to avoid overwriting images with the same name.

9.12 Create Recipe Save Code

This code saves the recipe first, then saves ingredients and instructions

```

ManagementSystem
RecipeManagementSystem.Controllers.RecipesCor
Create(Recipe recipe, IF

_context.Recipes.Add(recipe);
await _context.SaveChangesAsync();

for (int i = 0; i < ingredientNames.Length; i++)
{
    if (!string.IsNullOrWhiteSpace(ingredientNames[i]))
    {
        var ingredientName = ingredientNames[i].Trim();

        var ingredient = await _context.Ingredients
            .FirstOrDefaultAsync(x => x.Name == ingredientName);

        if (ingredient == null)
        {
            ingredient = new Ingredient { Name = ingredientName };
            _context.Ingredients.Add(ingredient);
            await _context.SaveChangesAsync();
        }

        var recipeIngredient = new RecipeIngredient
        {
            RecipeId = recipe.RecipeId,
            IngredientId = ingredient.IngredientId,
            Quantity = quantities.Length > i ? quantities[i] : ""
        };

        _context.RecipeIngredients.Add(recipeIngredient);
    }
}

```

Figure 17 The recipe must be saved first so that it gets a RecipeId.

9.13 Shopping List Aggregation Code

The shopping list is generated from meal plans.

```

18 var shoppingList = await _context.MealPlans
19     .Include(m => m.Recipe)
20     .ThenInclude(r => r!.RecipeIngredients)
21     .ThenInclude(ri => ri.Ingredient)
22     .SelectMany(m => m.Recipe!.RecipeIngredients)
23     .GroupBy(ri => ri.Ingredient!.Name)
24     .Select(g => new ShoppingListItemViewModel
25     {
26         IngredientName = g.Key,
27         Quantities = string.Join(", ", g.Select(x => x.Quantity))
28     })
29     .ToListAsync();
30
31 return View(shoppingList);
32
33 }

```

Figure 18 This is one of the main algorithms in the project. It collects ingredients from all planned recipes and groups them by ingredient name.

9.14 Rating Average Code

The recipe card shows the average rating.

```

42 @{
43     var avgRating = item.Reviews.Any() ? item.Reviews.Average(r => r.Rating) : 0;
44 }

```

Figure 19 This prevents an error when a recipe has no reviews.

9.15 Navigation Code

The navigation links allow users to move between Recipes, Meal Planner, and Shopping List.

```

28
29
30
31
32
33
34
35
36
37
38
39
40
41
<ul class="navbar-nav flex-grow-1">
  <li class="nav-item">
    <a class="nav-link text-dark" asp-controller="Recipes" asp-action="Index">Recipes</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-controller="MealPlans" asp-action="Index">Meal Planner</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-controller="ShoppingList" asp-action="Index">Shopping List</a>
  </li>
</ul>

```

Figure 20 A small mistake in this file caused the Recipes link to display incorrectly. The issue was fixed by adding the missing <a> tag correctly.

9.16 Back Button Code

Back buttons were added to improve navigation.

```

6
<a asp-controller="Recipes" asp-action="Index" class="btn btn-secondary">Back to Recipes</a>

```

Figure 21 This was added to the Meal Planner and Shopping List pages.

10. Testing and Validation

Testing was done manually by running the application in Visual Studio and checking each function [1].

10.1 Test Plan

Table 1

Test No.	Test Description	Expected Result	Actual Result	Status
1	Open Recipes page	Recipes Page must load	Page loaded	pass
2	Add Chicken Pasta recipe	Recipe must save	Recipe saved	Pass
3	Upload recipe image	Image must display	Image displayed	Pass
4	Add ingredients	Ingredients must show in detail	Ingredients displayed	Pass
5	Add instructions	Steps must show in order	Steps displayed	Pass
6	Search by recipe title	Matching recipe must show	Recipe found	Pass
7	Search by ingredient	Matching recipe must show	Meal appeared	Pass
8	Add meal plan	Meal must appear in planner	Meal appeared	Pass

9	Open shopping list	Ingredients from meal plan must show	Ingredients displayed	Pass
10	Add review	Review must show on recipe details	Review displayed	Pass
11	Edit recipe	Updated details must save	Recipe updated	Pass
12	Delete recipe	Recipe must be removed	Recipe removed	Pass

10.2 Recipes Used for Testing

The following recipes were added during testing:

1. Chicken Pasta
2. Beef Burger
3. Vegetable Rice
4. Banana Smoothie
5. Chicken Salad
6. Pancakes

These recipes helped test different categories such as Breakfast, Lunch, and Dinner.

10.3 Bug Tracking and Fixes

Table 2

Bug	Cause	Fix
Recipes link showed as text	Missing < in anchor tag	Corrected _Layout.cshtml
Could not return from Meal Planner	No back button	Added Back to Recipes button
SQL timeout error	SQL Server Express connection problem	Changed connection string to LocalDB
Shopping list empty	No meal plan added yet	Added meal plan first
Save button looked like it failed	Database connection timeout	Fixed database connection

11. Validation Against Project Requirements

Table 3

Requirement	Implemented?	Explanation
Add recipes with ingredients	Yes	Recipe form includes ingredient names and quantities
Step-by-step instructions	Yes	Instruction model stores steps
Upload recipe images	Yes	Images are saved in <code>wwwroot/images/recipes</code>
Search/filter by category/ingredients	Yes	Search checks title, category, and ingredient
Meal planning calendar	Yes	Meal plans store date, meal type, and recipe
Auto-generate shopping lists	Yes	Shopping list groups ingredients from meal plans
Rating and review system	Yes	Review model stores reviewer, rating, and comment
Many-to-many relationships	Yes	RecipeIngredient joins Recipe and Ingredient
Many-to-many relationships	Yes	Meal plans are shown by date in a table

12. Professional Considerations

12.1 Ethical Considerations

The app gotta handle user data right. Recipes and reviews can show usernames, so don't just expose people's private info everywhere. If the app grows later, add user accounts and privacy settings so folks can choose what gets shared.

12.2 Security Measures

The system uses validation attributes like [Required], [MaxLength], and [Range] to catch bad input, and it uses [ValidateAntiForgeryToken] on form posts to help stop CSRF attacks. These measures help keep dodgy form submissions out.

12.3 POPIA Compliance

Reviews store names, so the app should follow POPIA, only collect personal info you actually need and tell users why you're keeping their names. If you add login later, passwords must be hashed and personal data locked down properly.

12.4 Scalability

The system can be expanded later by adding:

1. User login and roles.
2. Recipe sharing.

3. PDF export.
4. Admin dashboard.
5. Ingredient price estimation.
6. Weekly meal plan calendar.
7. Email shopping list.

12.5 Maintainability

It's easy to keep working on because everything is split into models, controllers and views. The MVC setup means you can fix one thing or add features without messing up the whole app.

13. Natural Science, Data Flow and System Behaviour

The system works by getting user input, processing it, and saving it [3]. for example, when someone clicks Save Recipe, the browser sends the form data to the RecipesController. the controller checks if the model is valid. if the data is good, it saves the recipe into the database using Entity Framework Core [4].

the system uses boolean logic in alot of places:

if (ModelState.IsValid)

this checks if the user uploaded an image.

if (id == null) return NotFound();

this stops the system from looking for a record that aint there. these checks help the system respond right to different things users do.

14. Mathematical and Analytical Techniques

The system uses basic math and analytical stuff.

14.1 Rating Average

calculates average rating like this:

Average Rating = Total rating points / Number of reviews

this helps users see how good a recipe is.

14.2 Shopping List Grouping

the shopping list groups ingredients by name. this is aggregation. instead of showing the same ingredient over and over, it groups them and shows the quantities together. example:

Meal Plan:

Chicken Pasta uses Salt 1 teaspoon

Vegetable Rice uses Salt 1 teaspoon

Shopping List:

Salt: 1 teaspoon, 1 teaspoon

14.3 Algorithm Complexity

The search algorithm goes through each recipe and compares the search text with the title, category, and ingredients. if theres more recipes, the search might get slower. for a small student project this is fine. for a bigger system, database indexing could be added later.

15. Limitations

The system works, but it still has some limitations:

1. There is no user login system.
2. Recipe sharing is not fully implemented.
3. The meal planner is a table, not a full graphical calendar.
4. The shopping list does not automatically combine quantities mathematically.
5. Image upload does not check file type or file size.
6. The system does not yet export reports to PDF.
7. Reviews can be added without user authentication.

16. Future Improvements

Future improvements can include:

1. Add login and registration.
2. Add roles such as Admin and Normal User.
3. Add a real calendar layout.
4. Add PDF export for shopping lists.
5. Add image file type validation.
6. Add recipe sharing by link.
7. Add nutrition information.
8. Add cost estimation for ingredients.
9. Add cloud deployment.
10. Add mobile-friendly improvements.

17. Conclusion

The Recipe Management System got designed and built using ASP.NET Core MVC, Entity Framework Core, SQL Server LocalDB, Razor Views, and Bootstrap [3],[4]. Users can add recipes, upload pictures, put in ingredients, write cooking steps, search recipes, plan meals, make shopping lists, and add ratings and reviews. The project meets the main requirements for Project 4 cause it got the required models, many-to-many relationship, image upload, shopping list grouping, and meal planning features [1]. It also supports GA2 because it uses software engineering knowledge, MVC architecture, database design, validation, testing, and problem solving [1].

During development some problems came up like a database timeout and a navigation issue. These got fixed by changing the connection string to LocalDB and fixing the navigation code. This shows testing and debugging were important parts of the project. Overall the project is a working full-stack web app that shows the software development lifecycle from figuring out requirements to building and testing it.

References

- [1] University of Zululand, Department of Engineering, "5EEE491 Individual Project Complete Guide: GA2 Alignment and Requirements," Software Engineering 5EEE491, 2026.
- [2] University of Zululand, Department of Engineering, "Lecture 3 5EEE491: ASP.NET Core MVC and the Code-First Approach," Software Engineering 5EEE491, 2026.
- [3] Microsoft, "ASP.NET Core MVC Overview," Microsoft Learn.
- [4] Microsoft, "Entity Framework Core Documentation," Microsoft Learn.
- [5] Microsoft, "Razor syntax reference for ASP.NET Core," Microsoft Learn.
- [6] Republic of South Africa, "Protection of Personal Information Act 4 of 2013," Government Gazette, 2013.